Email: John.dixon@hull.ac.uk
Meetings: https://tinyurl.com/jdixonhull
Feedback: https://tinyurl.com/jdhullfeedback
Office: RBB-315

Concurrency Principles

Distributed Systems Programming

Dr John Dixon

UNIVERSITY of HULL

Overview

We are looking at these topics today:

- The basic principles and problems of concurrency
- Transactions/Processes/Threads and how they differ
- Transaction failures and problems with simultaneous access
- Serial equivalence
- Some of the tools we can begin to use to overcome the problems associated with concurrency

- In the next session we'll look at how we can overcome some of these problems

Differentiating Processes, Threads and Asynchronous Operations

- Process
  - Instance of a program
  - Management at the operating system level
  - Has its own address space
- Thread
  - Component of a process – may be more than one running concurrently
  - Managed by the process
  - Scheduling at the operating system level
  - Typically running on one core of the machine
  - Threads within a process share the same address space
- Async Operation (recap)
  - Non-blocking, allowing the thread to continue to operate whilst a remote resource is accessed
  - Only initiates the operation, then continues to run subsequent code
  - May or may not include completion discovery (e.g. handlers)
  - May be harder to implement timeout/error discovery

Interleaving

- Interleaving is a procedure whereby processes or threads are split into actions and the actions are executed in a sequential order. Often this is managed by the OS and gives the impression of simultaneous execution.

- Example:
  - Process P | Actions: p1, p2
  - Process Q | Actions: q1, q2

  - Possible interleaving
    - p1 → q1 → p2 → q2
    - p1 → q1 → q2 → p2
    - p1 → p2 → q1 → q2
    - q1 → p1 → q2 → p2
    - q1 → p1 → p2 → q2
    - q1 → q2 → p1 → p2

Multiple-cores means interleaving may not be necessary:

Core1	Core2
p1 → p2	q1 → q2
q1 → q2	p1 → p2

Would this interleaving be a good use of processor time?

Core1	Core2
p1 → q1	p2 → q2

UNIVERSITY of HULL

Principles of Concurrency

- Processes, threads and interleaving
  - Facilitates simultaneous access and synchronicity
  - Allow us to do more than one thing at a time – making use of multi-processor architectures and distributed hardware
  - Have a tendency to make operations more complicated and error-prone, especially data operations; so we need…

- Concurrency handling techniques for distributed applications
  - Transactions
  - Locking
  - Advanced Locking and Monitors
  - Optimistic concurrency control (OCC)
  - Timestamping

UNIVERSITY of HULL

Transactions

- A sequence of server operations guaranteed by the server to be atomic in the presence of multiple clients and server crashes

- Transactions have ACID properties:
  - **Atomicity**: transaction must execute completely or not at all
  - **Consistency**: transaction must move the system from one consistent state to another
  - **Isolation**: each transaction must be performed without interference from other processes
  - **Durability**: when successful, the transaction's effects are made permanent in the storage medium

Types of Transaction Failure

- Deliberate client aborts
  - Transaction cancelled by client before completion
  - ➢ Transaction is not committed
- Accidental client aborts
  - Client may crash, exit or become unresponsive
  - Connection may fail
  - ➢ Server may use an expiry time
  - ➢ Expired unsent transactions are not committed
- Server aborts
  - Transaction cancelled by server/manually before completion (e.g. for safety reasons)
  - Server may crash, fail or become overloaded
    - Server process will be replaced (eventually)
  - ➢ Some/all transactions are not committed
  - ➢ Client must decide how to proceed

Concurrency

- Distributed applications are inherently concurrent
  - Many clients using a single server
  - Client connected to several servers
  - Multiple peer-to-peer connections
  - Each endpoint is likely processing concurrently

- Multiple threads are efficient enough
  - Offer concurrent or seemingly concurrent access to resources (e.g. processor time)
  - Although there is overhead in creating new threads so they shouldn't be used when not useful

- Most common concurrency problems are with data because concurrent access to a fixed resource (e.g. a file) is difficult
  - Problems with database operations or other common data stores
  - Problems with multiple processes operating on a single set of data

Problems with Simultaneous Access

- Lost updates
  - Process 1 reads, modifies, writes data
  - Process 2 reads data before process 1 has written, modifies and writes
  - Changes made by process 1 are lost

- Inconsistent retrieval
  - Process 1 reads many records (perhaps to calculate an average)
  - Process 2 alters some records while process 1 is running
  - Process 1 produces the wrong result

- Dirty reads (not like that)
  - Process 1 (an update) is aborted half way through
  - Process 2 has already read values changed by process 1
  - Process 1 reverts it's changes due to it being cancelled
  - Process 2 is thus using incorrect data

Lost Updates

- To make a deposit into a bank balance, the following steps must take place:
  - Read current balance from database into local variable
  - Add deposit to local variable
  - Write local variable back to database

```
balance = RemoteBank.GetBalance()
balance = balance + 100
RemoteBank.SetBalance(balance)
```

- What if two deposits happen at once?

Lost Updates

- A customer's bank account contains £100. He receives two deposits, one for £10 and one for £50. The final total should be £160. But what happens if the two deposit processes overlap?

Time

Deposit 1	Memory 1	Deposit 2	Memory 2	Balance
Read	£100			£100
Add £10	£110	Read	£100	£100
Write	£110	Add £50	£150	£110
		Write	£150	£150
				£150

UNIVERSITY of HULL

Inconsistent Retrieval

- Happens when one process uses data which is being changed by another process

  E.g. A transfer of £100 is required from account a to account b. The total a + b stays at £800.
  A separate process tries to calculate the total funds in all accounts.
  What happens if the two processes overlap?

Time

Process 1 (transfer £100 from a to b)

Operation	temp	Balance a	Balance b
Read a to temp	£300	£300	£500
Sub £100	£200	£300	£500
Write a	£200	£200	£500
Read b to temp	£500	£200	£500
Add £100	£600	£200	£500
Write b	£600	£200	£600

Process 2 (total accounts)

Operation	Total	temp
Read b to temp	£0	£500
Add temp to total	£500	£500
Read a to temp	£500	£200
Add temp to total	£700	£200
	£700	

UNIVERSITY of HULL

Dirty Read

- Happens when one process uses data which has been changed but the change is cancelled
- E.g. A transfer of £100 is cancelled from account a to account b. A separate process tries to a transfer of £300 from account a to account b.. What happens if the two processes overlap?

Time

Process 1 (transfer £100 from a to b)

Operation	temp	Balance a	Balance b
Read a	£300	£300	£500
Sub £100	£200	£300	£500
Write a	£200	£200	£500
CANCEL			
Add £100	£300	£200	£500
Write a	£300	£300	£500

Process 2 (transfer £300 from a to b)

Operation	temp	Balance a	Balance b
Read a	£200	£200	£500
Sub £300	-£100	£300	£500
Write a	-£100	-£100	£500
…	…	…	…

UNIVERSITY of HULL

Avoiding Conflict

- There are many strategies for avoiding errors caused by simultaneous access

- <u>The simplest is to forbid simultaneous access</u>
  - However, often this isn't acceptable because this can severely limit the scalability and performance of our system

- So we need to employ Avoidance or Detection mechanisms to…
  - Ensure that data being used by one process cannot affect or be affected by another process
  - Or identify when data being used by one process has affected or been affected by another process

- This is usually handled by the DBMS (Database Management System) made transparent to the user
  - Although we may need to set up the database in such a way that it can manage avoidance/detection
- Or managed in the data access layer
  - E.g. Entity Framework

How can we start to avoid/detect issues?...

UNIVERSITY of HULL

Serial Equivalence

- How do we know if two transactions have the correct effect when they run concurrently (interleaved)?

- Often its easy for a developer to understand a serial program (where code is run top-to-bottom) but difficult to understand a multi-threaded application. Instead, compare the effects…

- Compare
  - Effect of running one transaction after the other (e.g. A then B then C then D)
  - Effect of interleaving them (e.g. A and B, then C and D)

- If the effect is the same, they have serial equivalence
- Serially equivalent transactions should be executed concurrently wherever possible for performance

- **In a nutshell: Concurrent transactions are said to be serially equivalent when their effect is the same as if they were carried out sequentially.**

UNIVERSITY of HULL

Serial Equivalence in IO

- Interactions between two processes
  - ***Read / Read***: no conflict. Effect of read operations does not depend on order of execution.
  - ***Read / Write***: possible conflict. Result depends on order of execution.
  - ***Write / Write***: conflict. Result depends on order of execution.

Serial Equivalence

- For serial equivalence between transactions…
  - Look at the order in which pairs of conflicting operations are executed
  - Two transactions are serially equivalent if this order is the same for all objects both transactions access

- Consider two transactions T1 and T2:
- T1:

```
x = read(i);
y = read(j);
write(k, x+y)
```

- T2:

```
write(i, 20);
write(j, 30)
```

- What are the two pairs of conflicting operations?
  - `x = read(i)` and `write(i, 30)`
  - `y = read(j)` and `write(j, 20)`

UNIVERSITY of HULL

Serial Equivalence

- Interleaved… Serially Equivalent Transactions:

T1	T2
	`write(i, 20)`
`x = read(i)`	
	`write(j, 30)`
`y = read(j)`	

i = 20
x = 20
j = 30
y = 30

- Serial Transactions:

T1	T2
	`write(i, 20)`
	`write(j, 30)`
`x = read(i)`	
`y = read(j)`	

i = 20
j = 30
x = 20
y = 30

Same effect so interleaving is serially equivalent

UNIVERSITY of HULL

Serial Equivalence

- Interleaved… Not Serially Equivalent Transactions:

T1	T2
`x = read(i)`	
	`write(i, 20)`
`y = read(j)`	
	`write(j, 30)`

x = 0
i = 20
y = 0
j = 30

- Serial Transactions:

T1	T2
	`write(i, 20)`
	`write(j, 30)`
`x = read(i)`	
`y = read(j)`	

i = 20
j = 30
x = 20
y = 30

Different effect so interleaving is not serially equivalent

UNIVERSITY of HULL

Serial Equivalence

- Consider two transactions T1 and T2:
- T1:

```
x = read(i);
y = read(j);
write(k, x+y)
```

- T2:

```
write(j, 30);
write(i, 20)
```

UNIVERSITY of HULL

Serial Equivalence

- Interleaved… Not Serially Equivalent Transactions (inconsistent ordering):

T1	T2
	`write(j, 30)`
`x = read(i)`	
	`write(i, 20)`
`y = read(j)`	

j = 30
x = 0
i = 20
y = 30

Clearly i and j are being accessed in an inconsistent order

- Serial Transactions:

T1	T2
	`write(j, 30)`
	`write(i, 20)`
`x = read(i)`	
`y = read(j)`	

i = 30
j = 20
x = 20
y = 30

Clearly i and j are being accessed in an consistent order

UNIVERSITY of HULL

Serial Equivalence

Serial equivalence doesn't *necessarily* mean <u>correct</u> ordering! You need to decide what is correct

- Serial Transactions:

T1	T2
`write(j, 20)`	
`write(i, 30)`	
	`x = read(i)`
	`y = read(j)`

j = 20
i = 30
x = 30
y = 20

- Also Serial Transactions:

T1	T2
`x = read(i)`	
`y = read(j)`	
	`write(j, 20)`
	`write(i, 30)`

x = 0
y = 0
j = 20
i = 30

After T1 and T2 have run `write(k, x+y)`

k = x+y = 50

k = x+y = 0

UNIVERSITY of HULL

Concurrency Control

- For safe concurrency, transactions must be ordered so that their effect is serially equivalent.

- Three common methods help to avoid conflict and ensure order is imposed on a shared concurrent system:
  - Locking
  - Optimistic concurrency control
  - Timestamp ordering

Concurrency Control

- Two key aspects to many forms of concurrency control:

- **Mutual exclusion**
  - Preventing object / resource from being accessed by more than one thread at once
  - Needed to prevent corruption of state in presence of multiple client updates
  - Implemented through locking

- **Synchronization**
  - Imposes order on interleaving of operations of multiple threads (usually for correctness)
  - Implemented using:
    - **Semaphores** (a variable – usually either binary or counting - used to control access to common resources),
    - **Condition variables** (a type representing a queue of processes waiting on a condition, also used to wake the next waiting process when the condition becomes true – useful when considering serial equivalence)
    - **Monitors** (usually use semaphores and condition variables to provide access to a resource and prevent multiple processes from accessing the resource simultaneously)

UNIVERSITY of HULL

Control & Synchronisation – Key terms for the next session

- ***Critical Section*** - a sequence of statements that appear to execute indivisibly, typically accessing shared resources
  - Code construct which marks a critical section of code that must be executed under mutual exclusion
- ***Mutual Exclusion*** - synchronization required to protect a critical section
- ***Locks*** - implement mutual exclusion and guard critical sections
- ***Conditional Synchronisation*** - Needed when a thread wishes to perform an operation which can only sensibly (safely) be performed if another thread has itself taken some other action or is in some defined state. E.g.
  - producer thread must not attempt to put data in buffer if it is full
  - consumer thread must not attempt to get data from buffer if it is empty
  - Is mutual exclusion necessary?

UNIVERSITY of HULL

Reflection

In this session we've looked at:

- The basic principles of concurrency
- Transactions/Processes/Threads
- Interleaving
- Transaction failures and problems with simultaneous access
- Serial equivalence
- The basics of how to start to solve these problems

In the next session we will be looking in more detail at solutions for concurrency-based problems

UNIVERSITY of HULL